# Bitácora de Registro Diario

Diseño e Implementación de un Analizador Léxico en Flex – Asignación 2

**Equipo:** Arenas Rojas Ivri Nitzani · Bautista Jiménez Israel · Hernández Torija Itzel Amairani · Juárez Jiménez Yarelli Sinai · López López Xochitl Nerina · Quiroz Martínez Maybelin

**Periodo:** 18 de septiembre – 25 de septiembre de 2026

## Inicio de la Asignación 2

(18 y 19 de septiembre de 2026)

| Integrante | 18/09/2026 | 19/09/2026 |
|---|---|---|
| **Arenas Rojas Ivri Nitzani** | Coordinó la reunión de arranque de la Asignación 2, presentó los requerimientos del analizador léxico en Flex y repartió los módulos de `lexer.l` entre los integrantes | Instaló y verificó Flex 2.6.4 y gcc, y definió la estructura de las tres secciones de `lexer.l` (definiciones, reglas y código de usuario) |
| **Bautista Jiménez Israel** | Revisó el formato de la tira de tokens de la primera entrega para reproducirlo en Flex (No línea, lexema y token) | Estudió el uso de `yytext` y `yyleng` dentro de las acciones de Flex y la escritura con `printf` y `fprintf` |
| **Hernández Torija Itzel Amairani** | Analizó los tipos de error léxico de la primera entrega y la forma de reportarlos desde las reglas de Flex | Estudió las condiciones de arranque exclusivas (`%x`) para recuperar el análisis después de un error |
| **Juárez Jiménez Yarelli Sinai** | Revisó la tabla de símbolos de la versión en Python para conservar el orden de aparición de los identificadores | Diseñó la estructura `ElementoTSimb` (nombre y apuntador al siguiente nodo) de la lista simplemente enlazada |
| **López López Xochitl Nerina** | Recopiló las expresiones regulares de la primera entrega para traducirlas a la notación de Flex | Tradujo las definiciones `letra`, `digito`, `identificador`, `nint` y `nfloat` a la sintaxis de Flex |
| **Quiroz Martínez Maybelin** | Recopiló las 42 palabras reservadas de la primera entrega agrupadas por categoría (tipo de dato, control, modificadores, estructura, entrada/salida) | Agregó `std`, `printf` y `scanf` a la categoría de salida de texto, completando las 45 palabras reservadas de `lexer.l` |

## Semana 3

(21 al 25 de septiembre de 2026)

| Integrante | 21/09/2026 | 22/09/2026 | 23/09/2026 | 24/09/2026 | 25/09/2026 |
|---|---|---|---|---|---|
| **Arenas Rojas Ivri Nitzani** | Programó `main()`: abre el archivo fuente y crea `tiraTokens.txt`, `tablaSimbolos.txt` y `errores.txt` | Definió la macro `YY_USER_ACTION` para llevar la columna de cada lexema y la regla del salto de línea | Integró los módulos de todos en un solo `lexer.l`, verificó la compilación con flex y gcc y agregó el cierre de los archivos | Redactó la introducción, el análisis y la compilación y ejecución en la terminal (Linux, macOS y Windows) del informe | Dirigió la reunión final, ensambló el informe y la distribución de responsabilidades y entregó la Asignación 2 |
| **Bautista Jiménez Israel** | Programó `escribirToken()` para palabras reservadas, operadores y delimitadores, que usan su lexema como token | Programó `escribirTokenValor()` para `id`, `nint`, `nfloat` y `literal` con el formato tipo.valor | Verificó la escritura simultánea en consola y en `tiraTokens.txt` y alineó las columnas de la tira de tokens | Comparó la tira de tokens de Flex con la de Python en los archivos de prueba y documentó la sección de tira de tokens | Participó en la revisión final del proyecto y validó la versión entregable del sistema |
| **Hernández Torija Itzel Amairani** | Programó `escribirError()` para reportar línea, columna, carácter y descripción en `stderr` y `errores.txt` | Definió las condiciones `RESTO_LINEA` y `RESTO_ARCHIVO` para cadenas, caracteres y comentarios sin cerrar | Agregó la regla de recuperación para caracteres no reconocidos y probó el módulo con `ErroresLexicos.cpp` | Documentó el manejo de errores en el informe con la tabla de recuperación y el autómata de condiciones de arranque | Participó en la revisión final del proyecto y validó la versión entregable del sistema |
| **Juárez Jiménez Yarelli Sinai** | Programó `insertar()`, que agrega al final de la lista, y `buscar()`, que la recorre secuencialmente | Programó `agregarSimbolo()` con memoria dinámica (`malloc`) sin duplicados y lo conectó con la regla `{identificador}` | Programó `imprimirTabla Simbolos()` en consola y en `tablaSimbolos.txt` y lo probó con `Variables.cpp` | Documentó la tabla de símbolos en el informe y ejecutó la batería de pruebas para obtener la tabla de resultados | Reorganizó la interfaz con el menú Analizadores (léxico Flex y AFD) y la pantalla principal con el logo, y actualizó el informe |
| **López López Xochitl Nerina** | Definió `escape`, `caracterCadena`, `literalCadena` y `literalCar` en la sección de definiciones | Escribió las reglas de espacios en blanco y de comentarios de línea y de bloque | Relacionó los estados de aceptación del AFD de JFLAP con las reglas de `lexer.l` y obtuvo las estadísticas con `flex -v` | Dibujó los autómatas de cada expresión regular y documentó la traducción de las expresiones regulares a Flex | Participó en la revisión final del proyecto y validó la versión entregable del sistema |
| **Quiroz Martínez Maybelin** | Escribió las reglas de palabras reservadas antes que `{identificador}` para respetar la regla de primera coincidencia | Escribió las reglas de operadores compuestos y simples, incluido `**` como un solo token, y de delimitadores | Validó la precedencia entre operadores con `Operadores.cpp` y `DobleApuntador.cpp` | Integró el panel Analizador en Flex en la interfaz (abrir, compilar y ejecutar) con sus tres ventanas de resultados | Preparó el ejecutable para Windows con win_flex y TinyCC integrados y participó en la revisión final |